Instruction for code execution

Setting up Ollama in Linux

Install ollama

curl -fsSL https://ollama.com/install.sh | sh

Setting up Ollama in Windows

Install ollama

Paste this in PowerShell

irm https://ollama.com/install.ps1 | iex

Or download from this link

https://ollama.com/download/OllamaSetup.exe

General instruction

Start the server

ollama serve

Command to check if the ollama is running

curl http://localhost:11434

Pull model

ollama pull gpt-oss:120b-cloud
ollama pull nemotron-3-nano:30b-cloud – for annotation
ollama pull gemma4:31b-cloud – for llm-as-a-judge

Check if the model is pulled

ollama list

To run the script

Make sure you are in a folder where there are these two file present

1. ollama_cloud_async.py
2. ollama_cloud_annotation_async.py
3. data_en.py

Option 1: Create a python-venv

For Linux

python3.x -m venv avenv

For Windows

python3.x.exe -m venv avenv

Use Python 3.11 or 3.12

Activate the venv

For Linux

source avenv/bin/activate

For Windows

.\avenv\Scripts\activate

Option 2: Create a conda-env

conda create -n py311 python=3.11

Activate env

conda activate py311

Debug

If not activated (there will be “(py311)” in the beginning of the line in the terminal), then go to the PowerShell and run this “*conda init powershell*”
Reopen VS Code and activate the conda env again.

Install requirements

pip install -r requirements.txt

Test the code

Usage: python ollama_cloud_annotation_async.py <perturbation_type:[1-7]>
Example python ollama_cloud_annotation_async.py 1

Usage: python ollama_cloud_async.py <model_name> <word_count_type:[1-3]> <word_count:[1-4]> <perturbation_type:[1-7]>

Example: python ollama_cloud_async.py gpt-oss:120b-cloud 1 1 1

Here ‘<>’ are the arguments needed to run the code. ‘[n-m]’ indicates the range of the arguments.
For word_count_type we have ['at least', 'at most', 'equal to']
And for word_count we have ['16', '128', '1024', '8192']
Finally, 7 people will have a total of 7 perturbation types.

Output:

The outputs will be in the directory you are running the code from.
You will have a file under ‘annotation/benign/’ folder after running the first code.
Then, after running the second code, you will see ‘output/benign/at-least/16/’ folder and a file in it.

‘benign’ is my (Shifat) decided perturbation type. Don’t worry about it now.

Debug

If you get the “unauthorized:401” error, type “*ollama login*” in the terminal and follow the login in the browser.

Full Task

Update the annotation code for your task

1. **Create an annotation prompt**

In the ‘ollama_cloud_annotation_async.py’, you will see ‘prompt_perturbation_benign’ in line 25. You need to come up with your own perturbation and make a prompt like this.
Discuss in the group to avoid overlap.

Add the prompt like this ‘prompt_perturbation_<type> = ”.

2. **Add the perturbation_type to the list**

In the main method, you will see a list of perturbation types.
perturbation_type_list = ['benign']
Add your type here
perturbation_type_list = ['benign', ‘your_type’]

3. **Add your type to the process**

In the process_prompt function, you need to add your type and prompt
original:

```
def process_prompt(prompt_whole, perturbation_type, model_name="kimi-k2-thinking:cloud"):
    if perturbation_type == "benign":
        prompt_annotation = prompt_perturbation_benign
```

Updated:

```
def process_prompt(prompt_whole, perturbation_type, model_name="kimi-k2-thinking:cloud"):
    if perturbation_type == "benign":
        prompt_annotation = prompt_perturbation_benign
     if perturbation_type == "your_type":
        prompt_annotation = prompt_perturbation_<your_type>
```

Test your updated version

python ollama_cloud_annotation_async.py 2

Update the execution code for your task

1. **Add the perturbation_type to the list**

In the main method, you will see a list of perturbation types.
perturbation_type_list = ['benign']
Add your type here
perturbation_type_list = ['benign', ‘your_type’]

Test your updated version

python ollama_cloud_async.py gpt-oss:120b-cloud 1 1 2

Now run for all instances

Before running: In both files, you will see 'df = df.iloc[:2]'. Comment this.

python ollama_cloud_annotation_async.py 2 – One time
python ollama_cloud_async.py <model_name> [1-3] [1-4] 2 – Multiple times for each model written below
So the second command will be ran 5(total model)*3(word count type)*4(word count)=60 times for one person
Execute in this sequence:
python ollama_cloud_async.py <model_name_1> 1 1 2
python ollama_cloud_async.py <model_name_1> 2 1 2
python ollama_cloud_async.py <model_name_1> 3 1 2

python ollama_cloud_async.py <model_name_2> 1 1 2
python ollama_cloud_async.py <model_name_2> 2 1 2
python ollama_cloud_async.py <model_name_2> 3 1 2

...

python ollama_cloud_async.py <model_name_1> 1 2 2
python ollama_cloud_async.py <model_name_1> 2 2 2
python ollama_cloud_async.py <model_name_1> 3 2 2

python ollama_cloud_async.py <model_name_2> 1 2 2
python ollama_cloud_async.py <model_name_2> 2 2 2
python ollama_cloud_async.py <model_name_2> 3 2 2

...

python ollama_cloud_async.py <model_name_1> 1 3 2
python ollama_cloud_async.py <model_name_1> 2 3 2
python ollama_cloud_async.py <model_name_1> 3 3 2

python ollama_cloud_async.py <model_name_2> 1 3 2
python ollama_cloud_async.py <model_name_2> 2 3 2
python ollama_cloud_async.py <model_name_2> 3 3 2

...

python ollama_cloud_async.py <model_name_1> 1 4 2
python ollama_cloud_async.py <model_name_1> 2 4 2
python ollama_cloud_async.py <model_name_1> 3 4 2

python ollama_cloud_async.py <model_name_2> 1 4 2
python ollama_cloud_async.py <model_name_2> 2 4 2
python ollama_cloud_async.py <model_name_2> 3 4 2

...

Models

For annotation: nemotron-3-nano:30b-cloud
For annotation evaluation: gemma4:31b-cloud
For experiment:

1. gpt-oss:120b-cloud
2. lm-5.1:cloud
3. qwen3.5:cloud
4. kimi-k2.5:cloud
5. Deepseek-v3.2:cloud

Extras

If anyone wants to use a server to run the code in the background, use ‘nohup’.
It should be present by default in most Linux systems. If not found, install ‘coreutils’ with your package manager (apt, yum, dnf, or pacman).
Example:
nohup python -u ollama_cloud_async.py gpt-oss:120b-cloud 1 1 1 2>&1 &